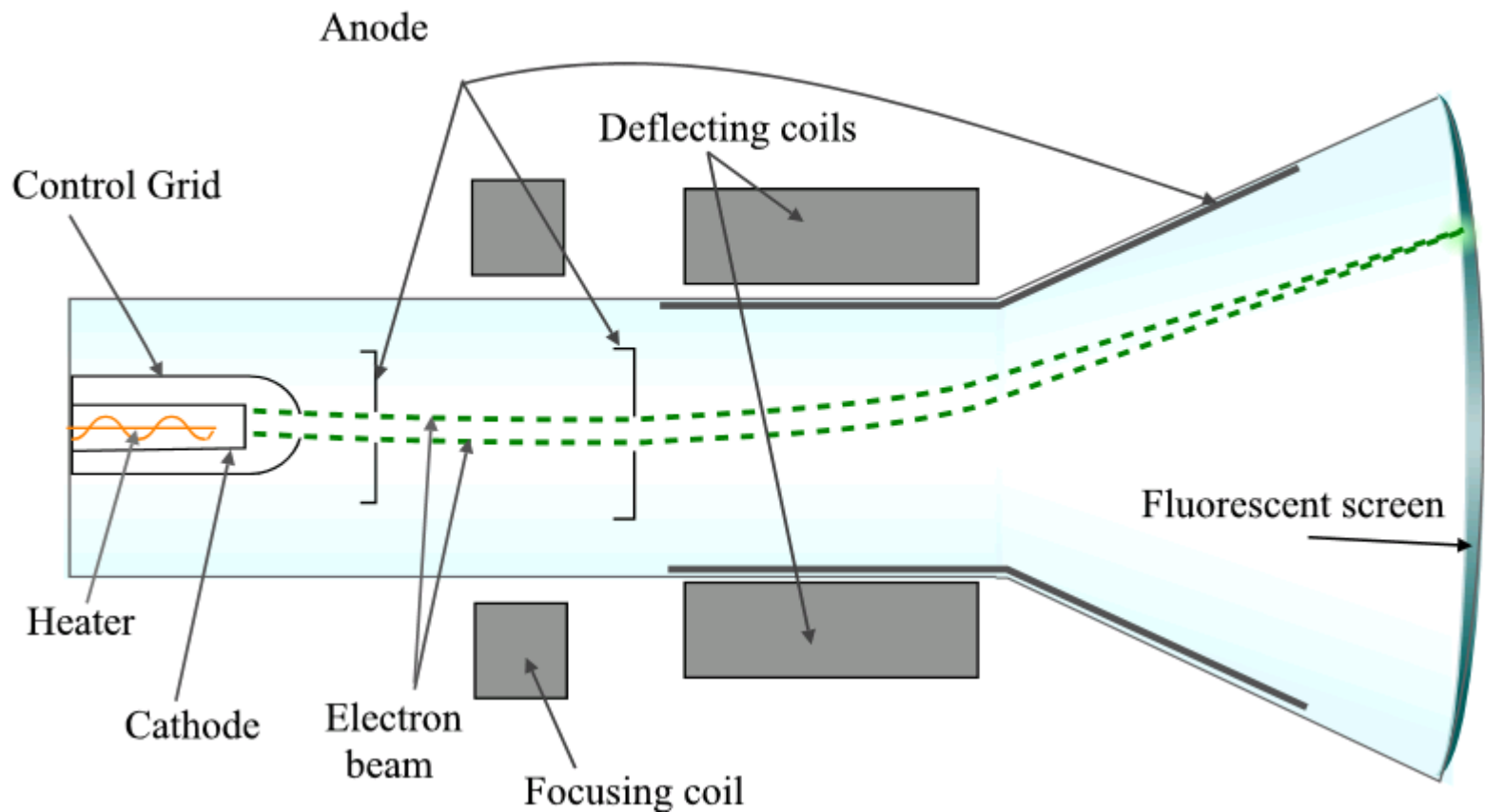


Rasterização

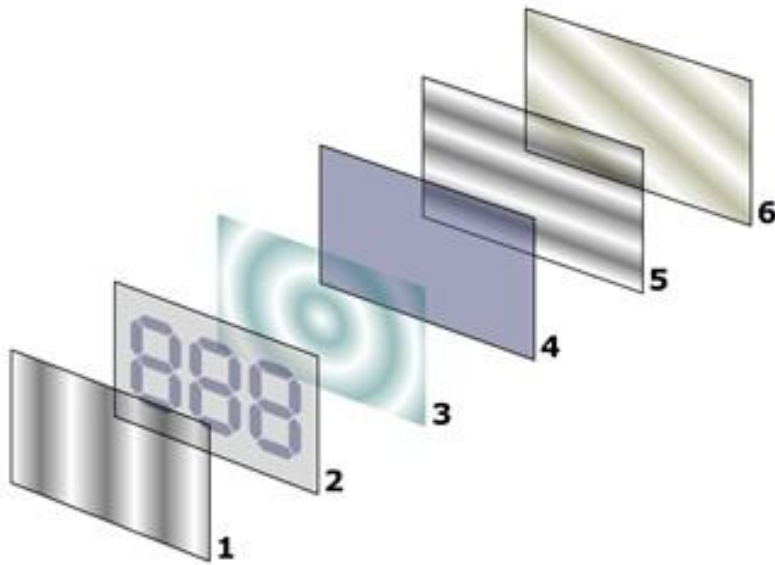
Prof. Paulo Bressan

Cathode-Ray Tube (CRT)



By Theresa Knott - en:Image:Cathode ray Tube.PNG, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=100143>

Liquid Crystal Display - LCD

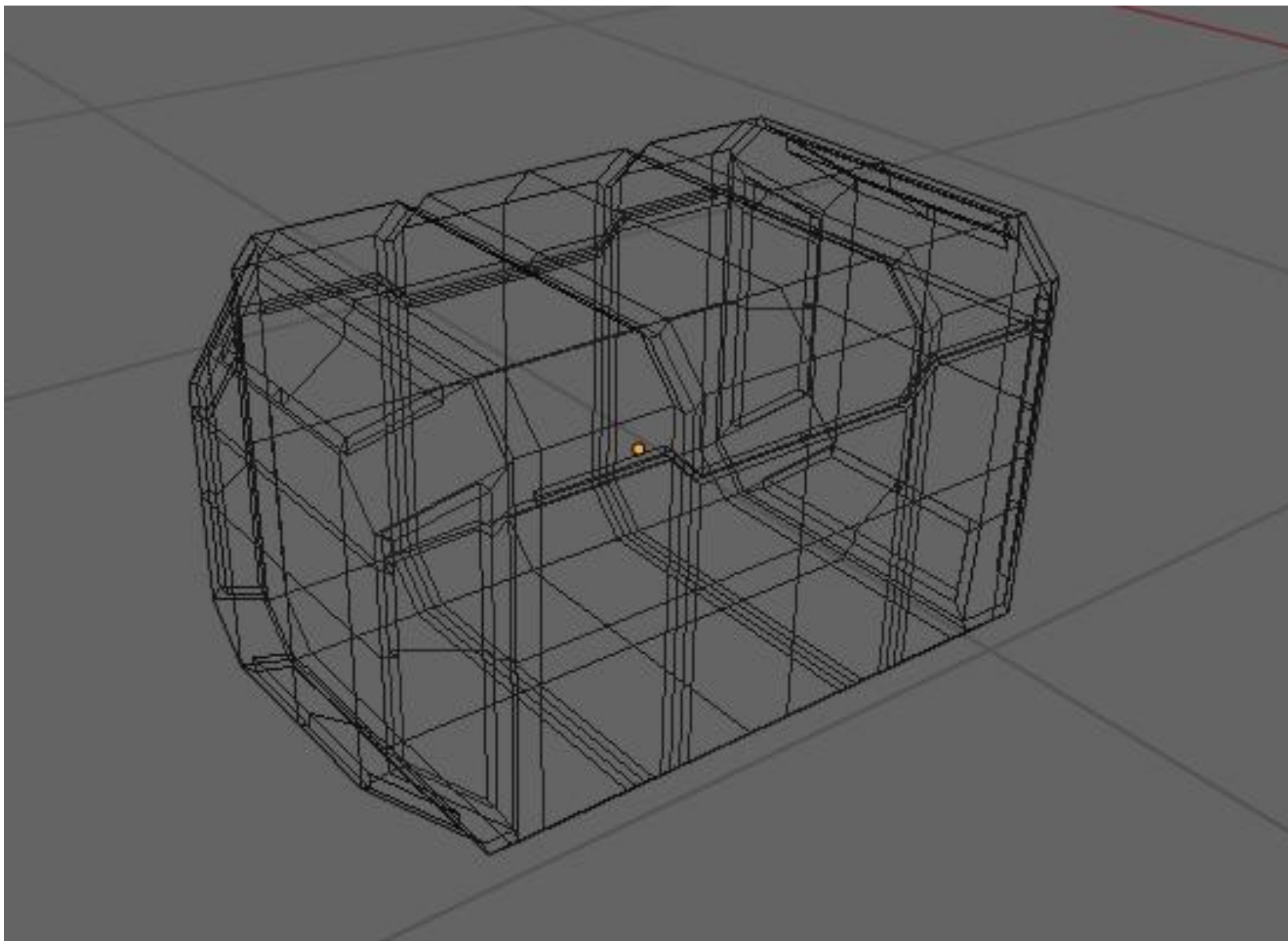


Wikipedia

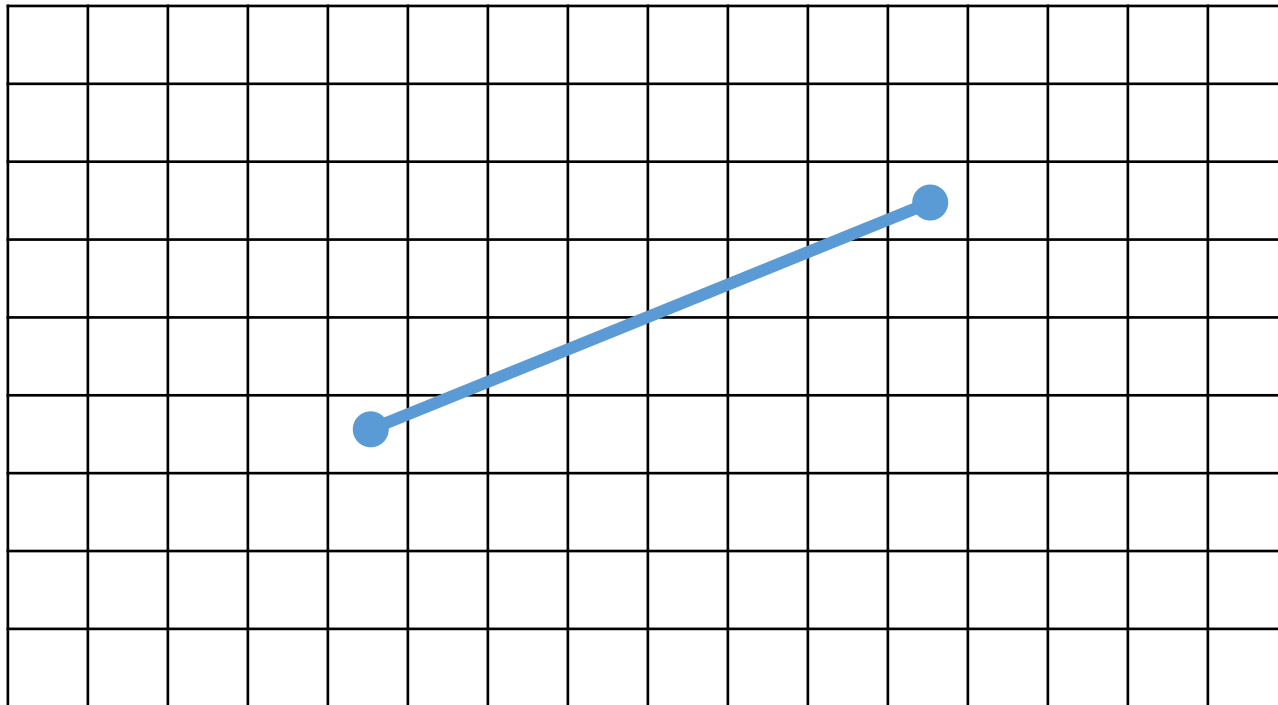
- 1.** Filtro vertical em película polariza a luz quando ela entra.
- 2.** Substrato de vidro com eletrodos de ITO. As formas desses eletrodos vão determinar o que aparece na TV quando ela está ligada. Cristas verticais são gravados na superfície juntamente com os cristais líquidos, em consonância com a luz polarizada.
- 3.** Cristal líquido.
- 4.** Substrato de vidro com película de eletrodo comum ITO com sulcos horizontais para se alinhar com o filtro horizontal.
- 5.** Filtro horizontal em película para bloquear ou permitir a passagem da luz.
- 6.** Superfície refletora para enviar a luz de volta ao telespectador.

Red Green Blue - RGB





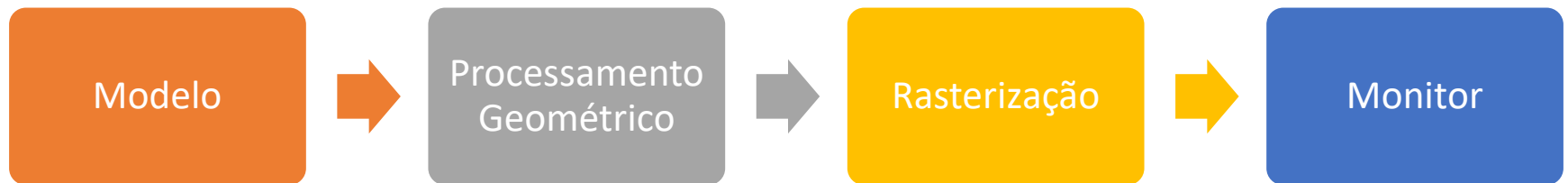
Quais pixels escolher?



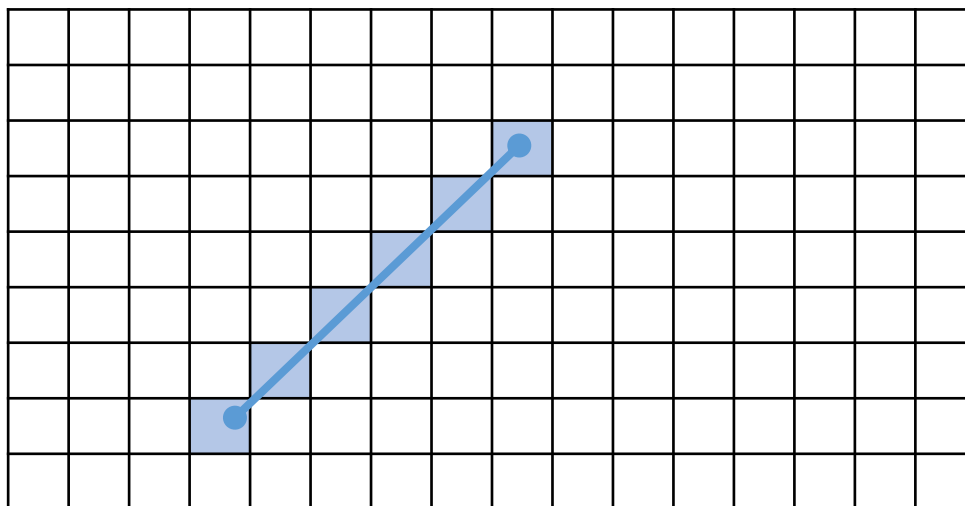
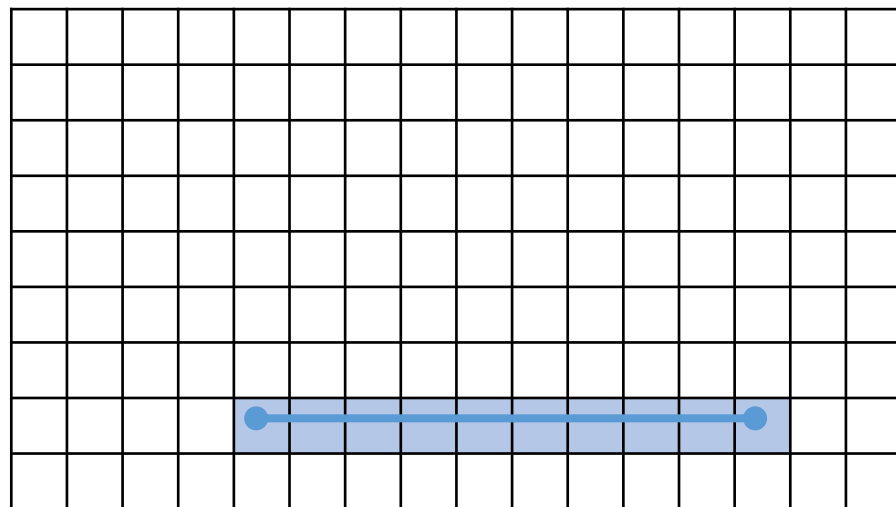
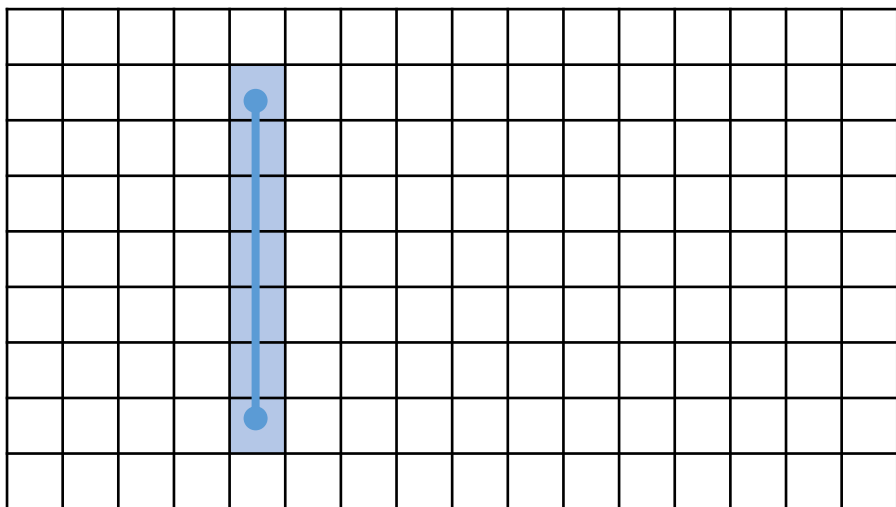
16 colunas
9 linhas

Resolução
16:9

Pipeline Básica



Algumas situações fáceis



Algoritmo DDA

- ▶ Digital Differential Analyzer
- ▶ Dispositivo eletromecânico para simular equações diferenciais
- ▶ Equação diferencial simples

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{dy}{dx} \quad \text{where} \quad 0 \leq m \leq 1$$

Algoritmo DDA

```
void line (int x1, int y1, int x2, int y2, int color)
{
    int x;
    float y, m;
    m = (y2 - y1) / (x2 - x1);
    y = y1;
    for (x = x1; x <= x2; x++) {
        writepixel(x, round(y), color);
        y = y + m;
    }
}

int round (float x)
{
    return (int)(x + 0.5);
}
```

Em termos de desempenho, qual o problema deste algoritmo?

Requer operações em ponto flutuante

Algoritmo de Bresenham (1965)

- ▶ Trabalha somente com inteiros.
- ▶ Dado um segmento de reta entre (x_1, y_1) e (x_2, y_2) , temos:

$$m = \frac{\Delta y}{\Delta x} = \frac{(y_2 - y_1)}{(x_2 - x_1)}$$

$$0 \leq m \leq 1$$

$$0^\circ \leq \theta \leq 45^\circ$$

$$y = m.x + q$$

$$y = \frac{\Delta y}{\Delta x}.x + q$$

$$\Delta x.y = \Delta y.x + \Delta x.q$$

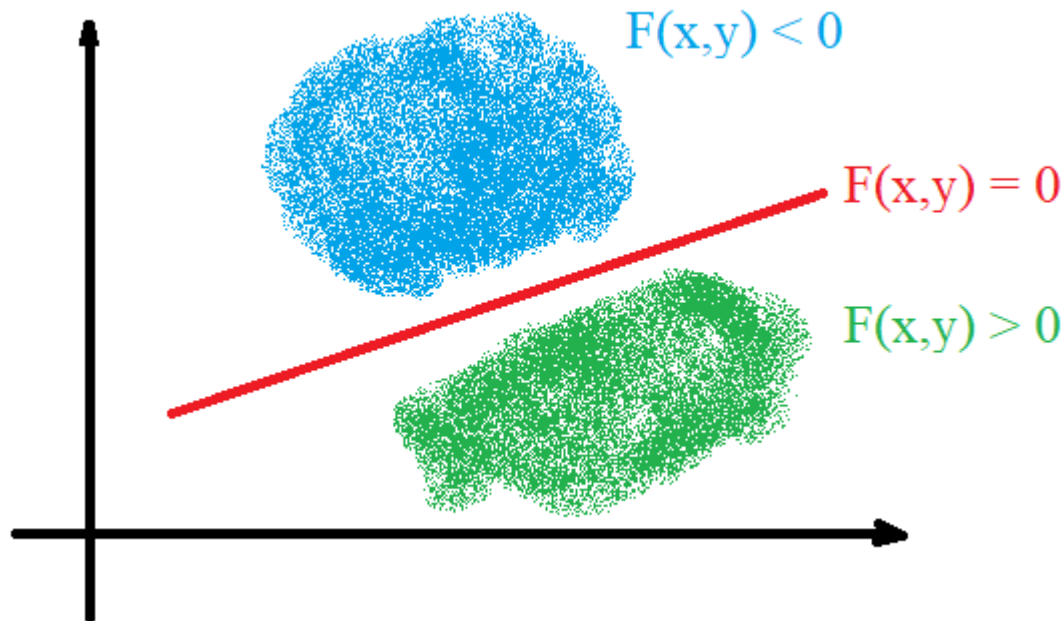
$$\Delta y.x - \Delta x.y + \Delta x.q = 0$$

$$F(x, y) = \Delta y.x - \Delta x.y + \Delta x.q$$

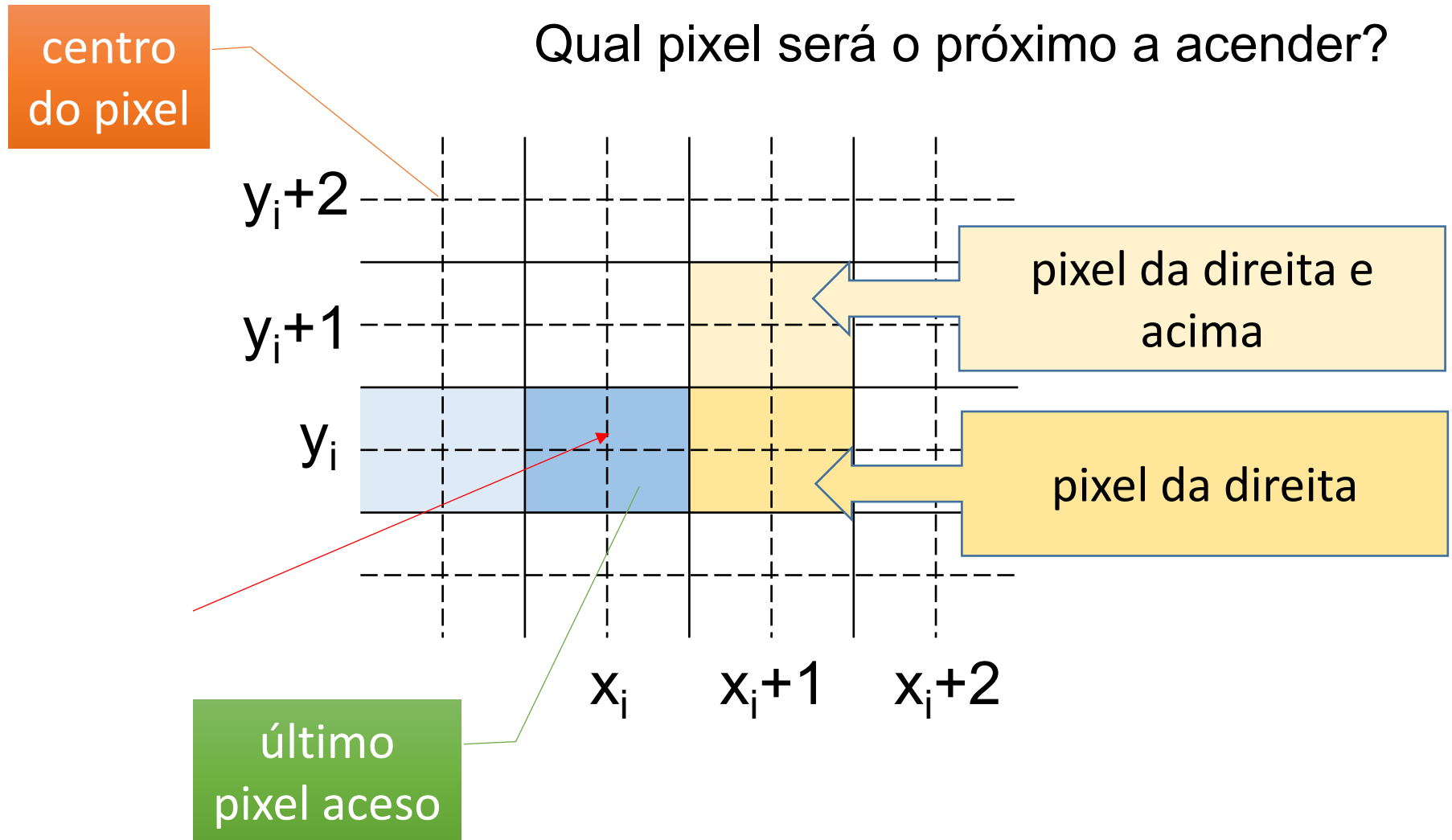
$$F(x, y) = a.x + b.y + c$$

Algoritmo de Bresenham (1965)

$$F(x, y) = \Delta y \cdot x - \Delta x \cdot y + \Delta x \cdot b \begin{cases} = 0, \text{ o ponto está sobre a reta} \\ > 0, \text{ o ponto está abaixo da reta} \\ < 0, \text{ o ponto está acima da reta} \end{cases}$$



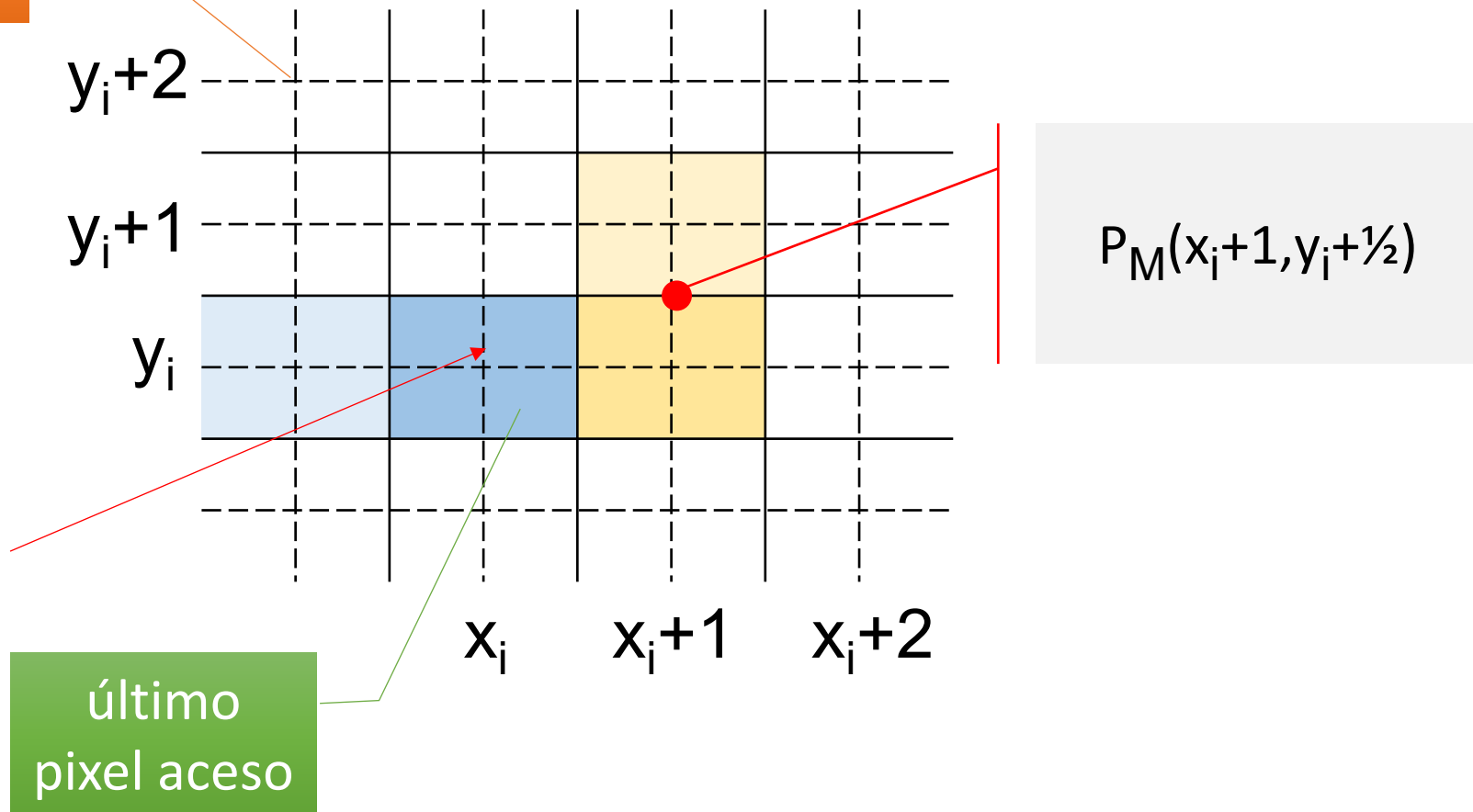
Algoritmo de Bresenham



Algoritmo de Bresenham

centro
do pixel

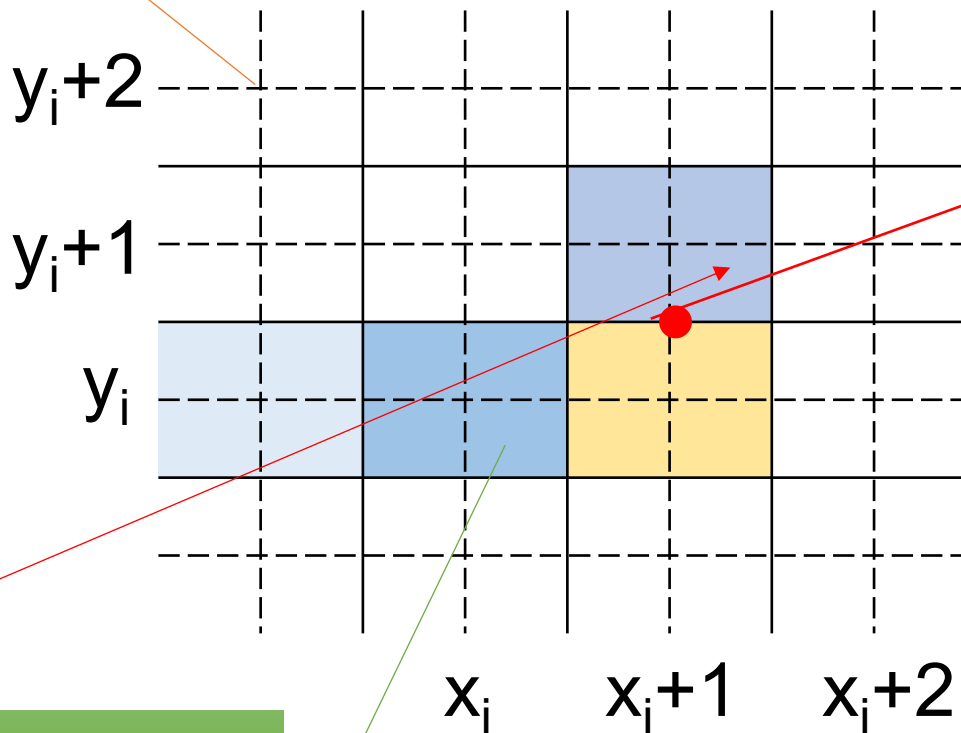
O ponto médio está abaixo ou acima da reta?



Algoritmo de Bresenham

centro
do pixel

Quando a reta passa acima do ponto médio, acender o pixel da direita e acima.



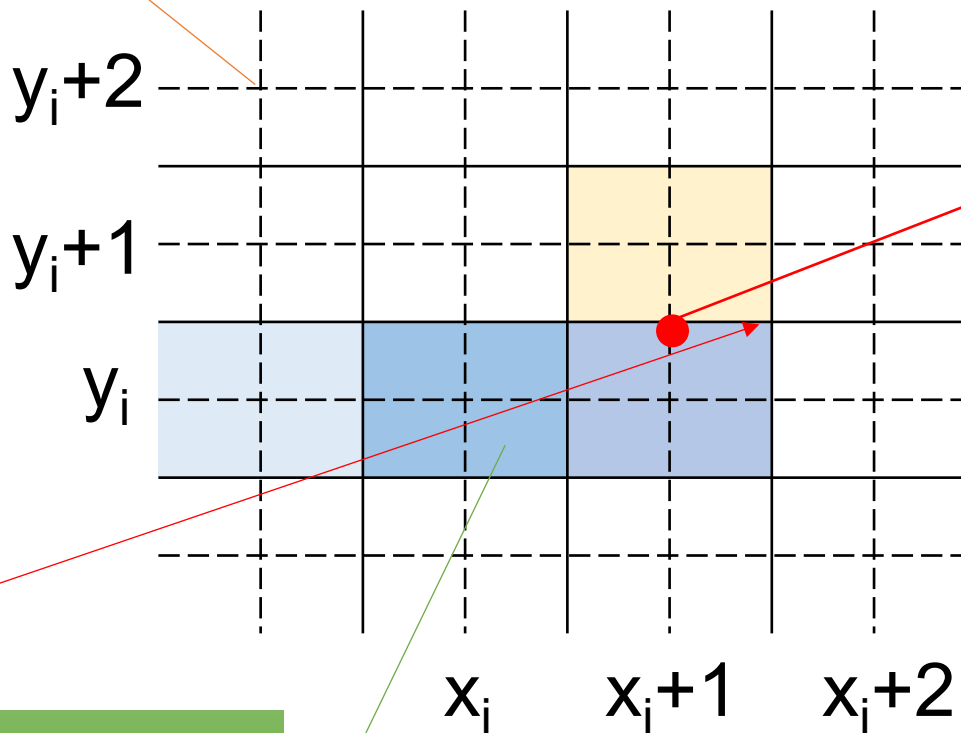
$$F(x_i+1, y_i+1/2) > 0$$

último
pixel aceso

Algoritmo de Bresenham

centro
do pixel

Quando a reta passa abaixo do ponto médio, acender o pixel da direita ao lado.



$$F(x_i+1, y_i+\frac{1}{2}) < 0$$

último
pixel aceso

Algoritmo de Bresenham

Seja d_i o resultado da função da reta sobre o ponto médio $M=(x_i+1, y_i+\frac{1}{2})$ no passo i :

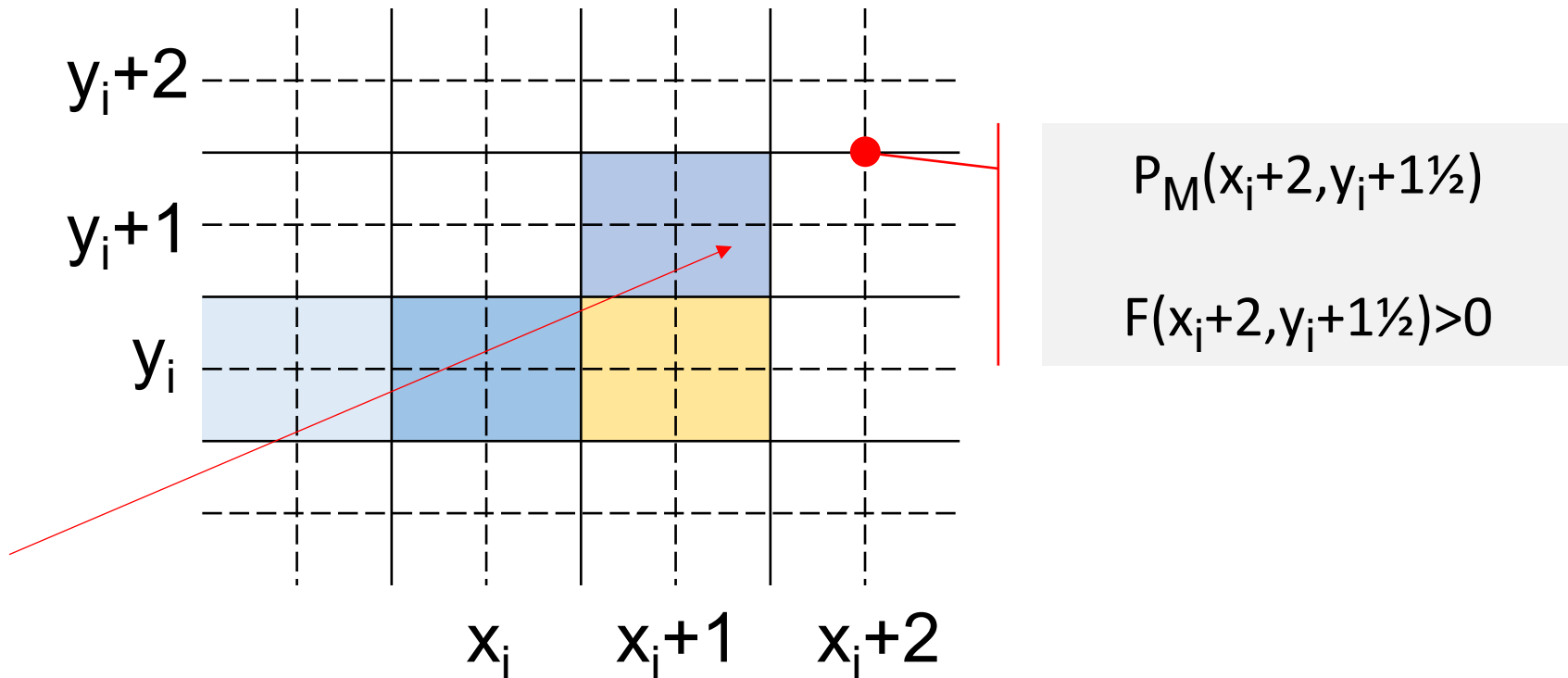
$$d_i = F(x_i + 1, y_i + \frac{1}{2}) = a \cdot (x_i + 1) + b \cdot (y_i + \frac{1}{2}) + c$$

Duas situações para analisar para o próximo passo:

1. se $d_i > 0$, então o ponto M está abaixo da reta, a reta está mais próxima do pixel da direita e acima.
2. se $d_i < 0$, então o ponto M está acima da reta, ou seja, a reta está mais próxima do pixel da direita e ao lado.

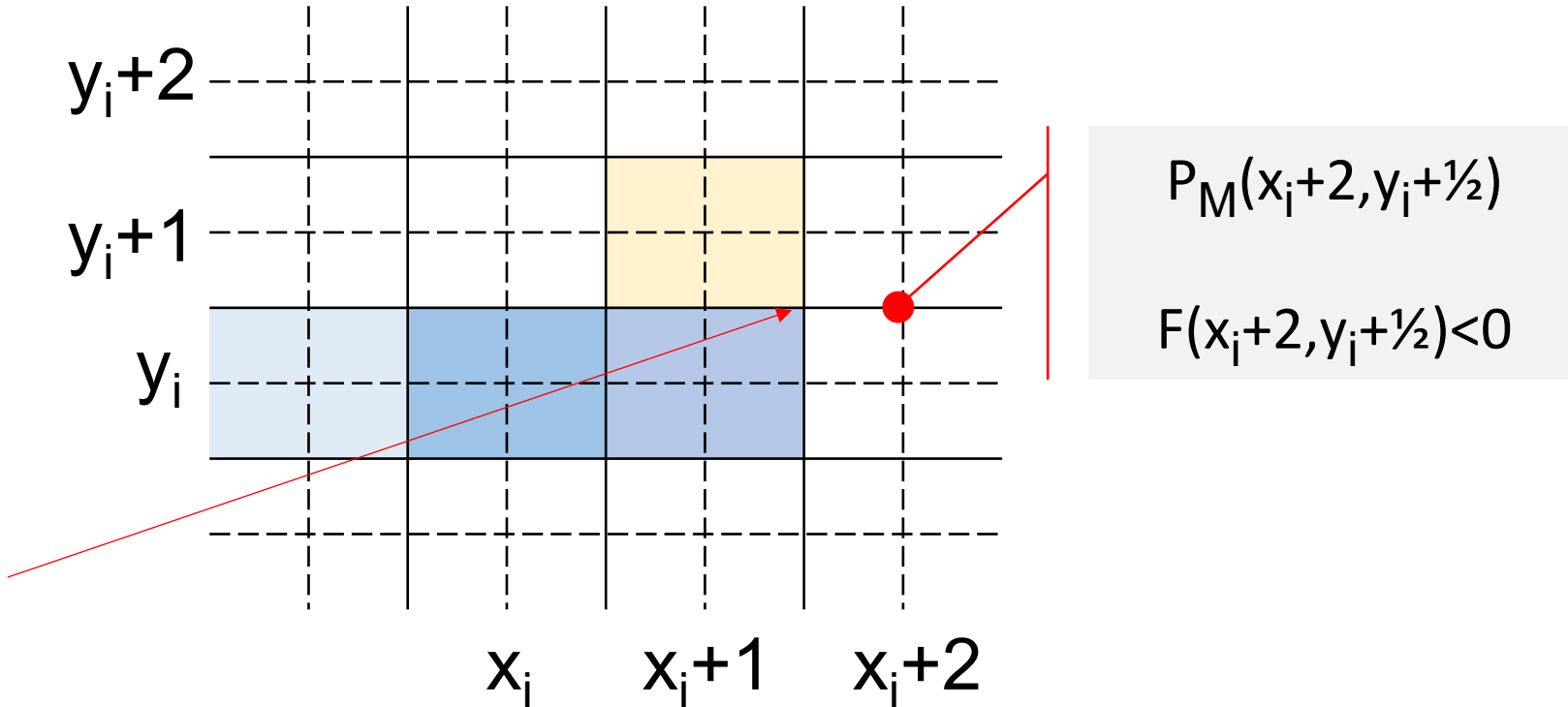
Algoritmo de Bresenham

Situação 1: no passo $i+1$, o ponto médio estará um linha acima e uma coluna à direita.



Algoritmo de Bresenham

Situação 2: no passo $i+1$, o ponto médio estará na mesma linha e uma coluna à direita.



Algoritmo de Bresenham

- No passo i:

$$d_i = F(x_i + 1, y_i + 1/2) = a \cdot (x_i + 1) + b \cdot (y_i + 1/2) + c$$

- No passo i+1:

- Ponto direita e acima:

$$d_{i+1} = F(x_i + 2, y_i + 1 1/2) = a \cdot (x_i + 2) + b \cdot (y_i + 1 1/2) + c$$

$$\Delta d_1 = d_{i+1} - d_i = a + b = \Delta y - \Delta x$$

- Ponto direita ao lado:

$$d_{i+1} = F(x_i + 2, y_i + 1/2) = a \cdot (x_i + 2) + b \cdot (y_i + 1/2) + c$$

$$\Delta d_2 = d_{i+1} - d_i = a = \Delta y$$

Algoritmo de Bresenham

- Qual será o primeiro valor de d ?

$$d_0 = F(x_0 + 1, y_0 + \frac{1}{2}) = a \cdot (x_0 + 1) + b \cdot (y_0 + \frac{1}{2}) + c$$

$$d_0 = a \cdot x_0 + b \cdot y_0 + c + a + \frac{b}{2}$$

$$d_0 = F(x_0, y_0) + a + \frac{b}{2}$$

$$d_0 = a + \frac{b}{2} = \Delta y - \frac{\Delta x}{2}$$

Pode resultar em
ponto flutuante

Algoritmo de Bresenham

- Multiplicando tudo por 2:

$$F'(x, y) = 2. (a.x + b.y + c)$$

$$d_0 = 2. \Delta y - \Delta x$$

$$\Delta d_1 = 2. (\Delta y - \Delta x)$$

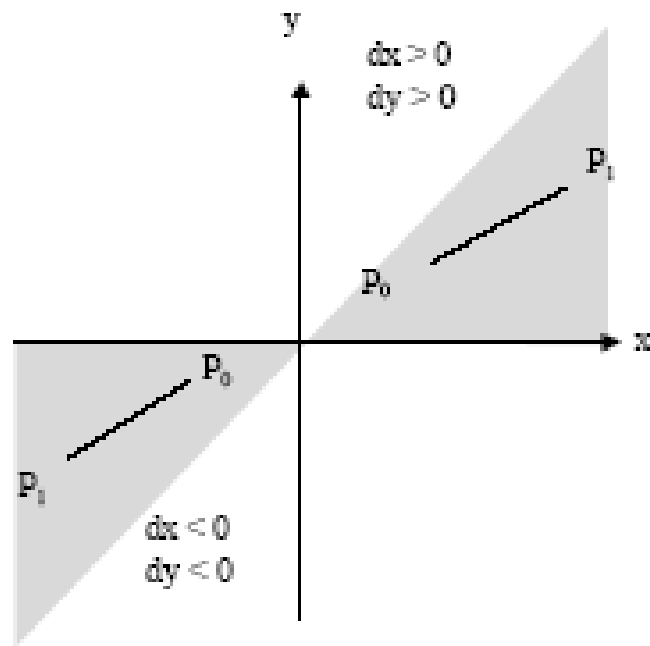
$$\Delta d_2 = 2. \Delta y$$

Algoritmo de Bresenham

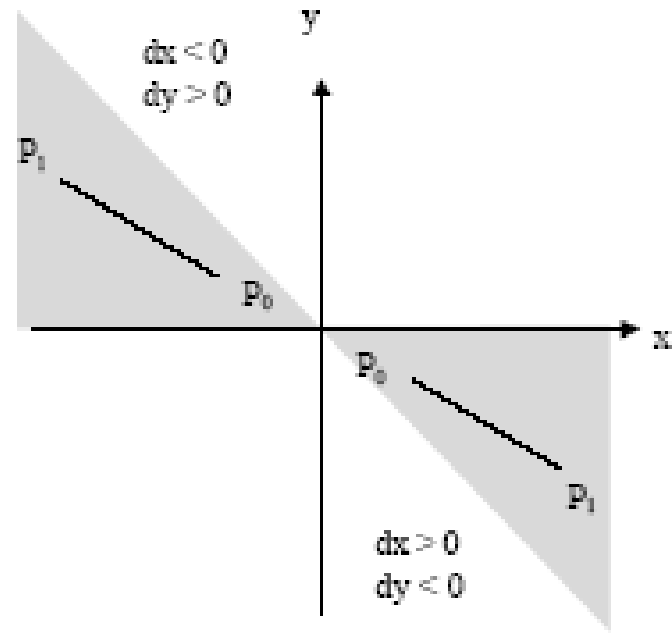
```
midpoint_line (int x1, int y1, int x2, int y2, int color){
    int dx, dy, incrE, incrNE, d, x, y;
    dx = x2 - x1;
    dy = y2 - y1;
    d = 2 * dy - dx;
    incrE = 2 * dy;
    incrNE = 2 * (dy - dx);
    y = y1;
    for (x = x1; x < x2; x++) {
        writepixel (x, y, color);
        if (d > 0) {
            d = d + incrNE;
            y++; }
        else
            d = d + incrE;
    }
    writepixel (x, y, color);
}
```

Em qual octante?

(a) $\underline{0 \leq m \leq 1}$

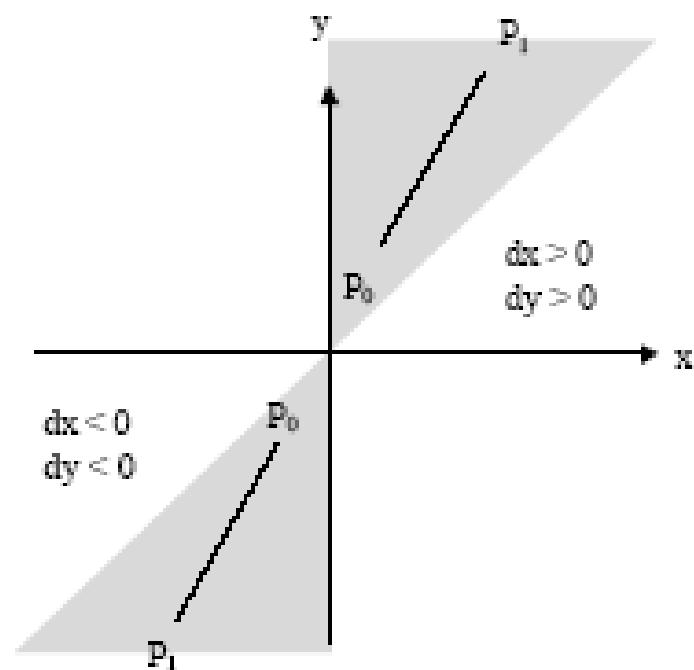


(b) $\underline{-1 \leq m \leq 0}$

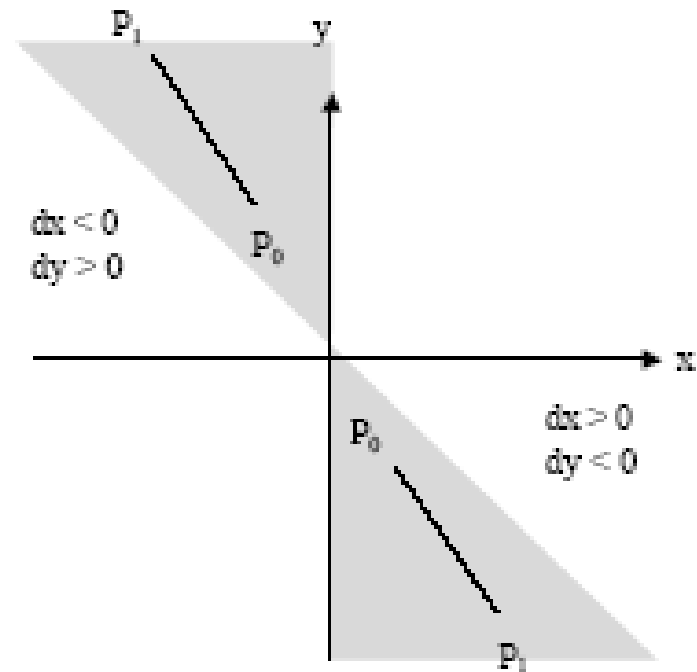


Em qual octante?

(c) $m > 1$



(d) $m < -1$



Em qual octante?

- Se $x_2 < x_1$
 - Trocar P1 com P2
- Se $y_2 < y_1$
 - $y_1 \leftarrow -y_1$
 - $y_2 \leftarrow -y_2$
 - Pintar pixel $(x, -y)$
- Se $|y_2 - y_1| > |x_2 - x_1|$
 - Repetir o algoritmo trocando “y” com “x”